Assignment 2: HTTP Server
CSE 130: Principles of Computer Systems Design

Due: January 31, 2025

Goals This assignment will provide you with experience building a system that uses client-server/strong modularity. Our learning objectives are: (1) practice implementing a client-server system, (2) practice the advantages of powerful abstractions, (3) practice implementing a large system that solves a large problem, (4) review string parsing, and (5) review memory management.

This assignment is more complex than Assignments 0 and 1. This doc is long out of necessity: we have a lot of things to explain! We encourage you first to read and annotate this document early (in the first few days that the assignment is released), then design your server with a "box-and-arrows" diagram (within the first week that the assignment is released), and only then start writing code.

How to submit:

1. Navigate to the `asgn2` directory in your repo.
2. Configure your `config.json` file (see Canvas or the README in the resources repo for more information).
3. `python3 -m autograder.run.submit [file1] [file2] ...`

Contents

1 Assignment Description **2**
- 1.1 Your Server's Execution . . . 2
- 1.2 HTTP Request Format . . . 4
- 1.3 HTTP Responses Format . . . 6
- 1.4 Method Semantics . . . 8
- 1.5 Additional Functionality . . . 9

2 Examples **10**

3 Rubric **11**

4 Resources **12**
- 4.1 Testing Tips . . . 12
- 4.2 Getting Started and Design Tips . . . 13
- 4.3 The Assignment2 Library . . . 15

1 Assignment Description

You will be building an HTTP server for this assignment. Your server should execute "forever" without crashing (i.e., it should run until a user types `CTRL-C` on the terminal). Your server will create, listen, and accept client connections that arrive on a *port*. Your server's task is to process the bytes clients send using the HTTP protocol. A key task in this assignment is building a resilient server for malformed (or even malicious) clients: *no matter what a client sends your server, your server should not crash!*

Your server should take a single, required command line argument, an `int`, named *port*. Note: you may need to run your server with `sudo` if using a port number between 1 and 1023. In other words, your server should be started by specifying the following command from the `asgn2` directory of your repository:

```
./httpserver <port>
```

1.1 Your Server's Execution

This section provides an overview of the tasks that your server should perform. First, your server should create structures needed to listen to incoming connections. Then, it should repeatedly accept a new client connection, process it, and close that connection. Note that this should occur one connection at a time (i.e., it should accept and process its first connection before it begins processing the second connection). We outline the requirements for each of these steps below:

Listening for Connections

First, your server should create a *socket*, bind the socket to a `port`, and make it listen for incoming connections. The helper library will include a struct definition, `struct Listen_Socket` (and an equivalent typedef `Listen_Socket_t`), and function to create (`ls_new`) and delete (`ls_delete`) a listen socket[1]. If the `port` passed to `httpserver` is invalid (not an integer between 1 and 65535) or if `httpserver` cannot bind to the provided `port`, then your `httpserver` should produce the message "'`Invalid Port\n`"' to `stderr` and exit with a return code of 1. Check out the return value of `listen_new`, you might be able to use it to help you with this task.

Accepting Connections

Your server should repeatedly accept connections made by clients to the `port`. Linux represents a connection with an external entity (in this case, a client) using the `socket` abstraction. We will provide you with a function, `ls_accept`, that blocks execution until a new incoming connection is made to the socket, `sock`, and then returns a new socket for the new connection.

After accepting a connection, your server should process the data that the client sends on that connection following the HTTP protocol. To do this, your server will need to read bytes sent by the client and send bytes to the client. The `socket`, created when your server accepts the new connection, makes this task possible. You should think of a socket as equivalent to the file descriptors that you learned about in the last assignment. Each socket, `sock`, it is an integer. Your server can read bytes in the order that they were written from the other side of a socket (i.e., the client) by calling `read` with `sock` as the `fd` argument to the call. Your server can write bytes in the order that they were written from the other side of the socket (i.e., the server) by calling `write` with the socket as the `fd` argument to the call.

Processing Connections

Your server should process a simplified subset of the HTTP Protocol[2]. In particular, your server should support two types of HTTP operations: `GET` and `PUT`. Below, we describe the HTTP request format that your server should be able to handle, the HTTP response format that your server should send to clients as a response, and then describe the processing for `GET` and `PUT` commands.

[1] Note: We encourage you to use our library functions, but you don't *have* to do so if you'd rather build everything yourself.

[2] The curious reader can find the whole HTTP Protocol at Request For Comments: 2616 (abbreviated RFC 2616). We've made several simplifications, though, so follow our instructions rather than the RFC.

Closing Connections

After your server finishes processing a request for a connection, your `httpserver` should close the connection. You can call `close` on a socket like you would any other file descriptor. Note: each connection will contain at most one valid request; if it contains extra bytes after a valid request, your server should ignore those bytes (no need to send an additional response!). Also: before closing each connection, your server should read all of the bytes that were sent by the client, regardless of whether the client sent a valid request in their connection.

1.2 HTTP Request Format

`GET` and `PUT` requests share the same request format (i.e., a valid `GET` request and a valid `PUT` request contain the same fields). Below we show the format, identifying those that are required and those that are Optional. Then, we describe each field in detail. Note, that each field is separated, without any additional whitespace or characters, by the sequence `\r\n`. This sequence is the sequence of the single character '`\r`' followed by the character '`\n`', **NOT** the sequence of four characters '\', followed by 'r', followed by '\', followed by 'n':

request	::=	*request-line*	(required)
		header-field-list	(optional, repeated)
		empty-line	(required)
		message-body	(optional)

You may assume a limit on the size of the first part of a valid request. The characters of the request, starting with the *request-line* and extending up to and including the *empty-line*, will not exceed 2048 characters. That is, the part of the request before any *message-body* will not exceed 2048 characters.

Below, we describe each of the fields of a request in detail. We use grammar rules following the regular expression syntax of `regex.h` from the 'C' standard library. In particular, anything written in `fixed-width font` can be used directly in `regcomp()`, one of the functions provided by `regex.h`.

Within these regular expressions, `\r` and `\n` represent single characters for carriage return and new line. Square brackets represent a single character within a set of characters or range, e.g., `[a-z0-9]` can be any single alphanumeric characters, while `[abc123]` can be an 'a', 'b', 'c', '1', '2', or '3'. A string in curly brackets represents a limited repetition of the preceding item, e.g., `{1,8}` indicates that the previous entity is repeated 1–8 times. In a regular expression, a period can match any letter. Below (and, in `regex.h`), we use the text '`\.`' to represent the actual '`.`' character. All other letters in `fixed-width font` represent their literal letter. Finally, we use '␣' to represent a space in each of the formats (so, other whitespace in the formats).

request-line Every valid request includes exactly one *request-line* with the following format, where each field (*method*, *uri*, and *version*) is a case-sensitive sequence of characters.

request-line	::=	*method*␣*uri*␣*version*`\r\n`	
method	::=	`[a-zA-Z]{1,8}`	
uri	::=	`/[a-zA-Z0-9.-]{1,63}`	
version	::=	`HTTP/[0-9]\.[0-9]`	(`\.` represents a single period)

- A valid *method* contains one to eight characters from the character range `[a-zA-Z]`. Your server only needs to implement (i.e., perform the semantics) of `GET` and `PUT`. (The distinction between "treats as valid" and "implements" will be clear after the Responses section below).
- A valid *uri* starts with the character '/', includes between 2 and 64 characters (including the '/'), and except for the leading '/', only includes characters from the character set `[a-zA-Z0-9.-]`. Each *uri* matches the file within the directory in which the `httpserver` is running. For example, the *uri* `/foo.txt` matches the file `foo.txt` from the folder in which the `httpserver` is running. Your server should only perform the semantics of `GET` and `PUT` on requests that include a valid *uri*.
- A valid *version* starts with `HTTP/` and includes a version number [0-9].[0-9]. Your `httpserver` should only implement version `1.1`, so it should only perform the semantics of `GET` and `PUT` requests that include a `version` equal to `HTTP/1.1`.

header-field-list Valid requests include a list of zero or more *header-field*s after the *request-line*.

header-field-list	::=	*header-field**	(* means zero or more)
header-field	::=	*key*`:`␣*value*`\r\n`	
key	::=	`[a-zA-Z0-9.-]{1,128}`	
value	::=	`[ -~]{1,128}`	(printable ASCII characters)

- The *key* ends with a ':' character. A valid *key* will be at least 1 character, at most 128 characters, and only contain characters that are alphanumeric, `.`, or `-`.
- A valid request's *value* will contain 1 to 128 characters. Each character will be in the "printable" set (`[ -~]{1,128}`[3]).
- Valid requests end each *header-field* with the sequence `\r\n`. A list of *header-field*s will be followed by *empty-line*. (Essentially, regardless of how many *header-field*s a request contains, the list will terminate with the sequence `\r\n\r\n`.)

empty-line A list of *header-field*s will be followed by *empty-line*.

empty-line ::= `\r\n`

The `\r\n` that is at end of the final *header-field*[4] will be followed by the `\r\n` of the *empty-line*, creating a final sequence before any *message-body*: `\r\n\r\n`.

message-body Valid `PUT` requests must include a *message-body*; valid `GET` requests will not include a *message-body* (or we can say that it has a *message-body* of zero bytes). Valid requests that include a *message-body* will also include a header, with a *key* of `Content-Length`, whose *value* will indicate the number of bytes in the *message-body*.

message-body ::= `.*` (zero or more bytes)

In summary, here is an example of a valid `PUT` request to the URI, `foo.txt`:

```
PUT /foo.txt HTTP/1.1\r\nContent-Length: 12\r\n\r\nHello world!
```

A valid `GET` request to the URI, `bar.txt`:

```
GET /bar.txt HTTP/1.1\r\n\r\n
```

And, another valid `GET` request to the URI, `bar.txt`:

```
GET /bar.txt HTTP/1.1\r\nUser-Agent: Curl\r\n\r\n
```

[3] A space is not considered printable by some, but we are including it
[4] or the *request-line* if there is an empty *header-field-list*

1.3 HTTP Responses Format

Your `httpserver` must produce a response for each request, regardless of whether the request is valid or not. Your response must follow the grammar:

response ::= *status-line*
header-field-list
empty-line
message-body

status-line The *status-line* indicates the type of response to the request. It consists of three fields.

status-line ::= *http-version*␣*status-code*␣*status-phrase*`\r\n`
http-version ::= `HTTP/1.1`
status-code ::= see Table 1
status-phrase ::= see Table 1

`httpserver` must always produce the *http-version* string `HTTP/1.1` regardless of the *http-version* provided in the request. A response with a *status-code* in the 200s indicates a successful response, in the 400s indicates an erroneous response, and in the 500s indicates an internal server error. Table 1 lists the *status-code*s that `httpserver` should produce, their associated *status-phrase*s that your server should produce in the response's status line, the message body that your server should produce as a part of the response, and when it should use each code.

status-code	*status-phrase*	*message-body*	**Usage**
200	OK	OK\n	When a method is successful
201	Created	Created\n	When a URI's file is created
400	Bad Request	Bad Request\n	When a request is ill-formatted
403	Forbidden	Forbidden\n	When a URI's file is not accessible
404	Not Found	Not Found\n	When the URI's file does not exist
500	Internal Server Error	Internal Server Error\n	When an unexpected issue prevents processing
501	Not Implemented	Not Implemented\n	When a request includes an unimplemented method
505	Version Not Supported	Version Not Supported\n	When a request includes an unsupported version

Table 1: List of HTTP status that your `httpserver` should support.

header-field-list The *status-line* should be followed by one or more *header-field*s. A response's *header-field* has the same format as the request header fields:

header-field-list ::= *header-field**
header-field ::= *key*`:`␣*value*`\r\n`
key ::= `[a-zA-Z0-9.-]{1,128}`
value ::= `[ -~]{1,128}` (printable ASCII characters)

The server should produce the header-field `Content-Length`, whose value should be equal to the size of the *message-body*. Your server should end each *header-field* with the sequence `\r\n`.

empty-line The *header-field* will be followed by *empty-line*.

empty-line ::= `\r\n`

The `\r\n` that is at end of *header-field* will be followed by the `\r\n` of *empty-line*, creating a final sequence before the *message-body*: `\r\n\r\n`.

message-body `httpserver` should produce a *message-body* with each response, whose size, in bytes, is equal to the value identified in the response's `Content-Length` header. Your server should produce the *message-body* indicated in Table 1 for each request, *except for valid `GET` requests*. We describe the correct *message-body* for valid `GET` requests in the section on **Methods**.

message-body ::= zero or more bytes of any value

1.4 Method Semantics

You must implement two HTTP methods, `GET` and `PUT`.

GET A `GET` request indicates that the client would like to receive the content of the file identified by the `URI`. If a request is valid and specifies a `URI` that is resident in the directory in which `httpserver` is executing, then `httpserver` should produce a response that...

- has a *status-code* of `200`
- has a *message-body* that includes the current state of the file pointed to by *uri*. Note, this may be an empty file!
- has a `Content-Length` that indicates the number of bytes in the file.

For all other requests (include those that are valid but where the *uri* indicates a non-existent file), your server should produce a *status-code*, *message-body*, and `Content-Length` based upon Table 1.

PUT A `PUT` request indicates that the client would like to update or replace the content of the file identified by the *uri*. There are two cases: If a valid `PUT` request's URI points to a **file that does not yet exist**, `httpserver` should...

- create the file
- set the file's contents equal to the *message-body* in the request
- produce a response with a *status-code*, *message-body*, and `Content-length` for the *status-code* `201` based upon Table 1.

If a valid `PUT` request's URI points to a **file that already exists**, `httpserver` should...

- replace the file's contents with the *message-body* in the request
- produce a response with a *status-code*, *message-body*, and `Content-length` for the *status-code* `200` based upon Table 1.

For all other requests, your server should produce a *status-code*, *message-body*, and `Content-Length` based upon Table 1.

1.5 Additional Functionality

In addition to supporting the methods listed above, your project should meet the following functionality and limitations:

- `httpserver` should produce responses with the appropriate status code (see `Usage` in Table 1).
- `httpserver` should not have memory leaks.
- `httpserver` should handle connections that do not send data.
- `httpserver` should not leak any file descriptors.
- `httpserver` should use at most 10 MB of memory regardless of input.
- `httpserver` must be reasonably time efficient.
- `httpserver` should never crash (e.g., it should never `segfault`).
- `httpserver` must be written using the 'C' programming language (*not C++!*).
- `httpserver` cannot use a `FILE *` functions, such as `fread`, `fwrite`, or variants of `put` and `get`. This includes: `fwrite`, `fread`, `puts`, `fputs`, `putc`, `fputc`, `putc_unlocked` `putchar`, `putchar_unlocked`, `putw`, `getc`, `fgetc`,`getc_unlocked`, `gets`, `fgets`, `getchar`, `getchar_unlocked`, and `getw`.
- `httpserver` cannot use functions, like `system(3)`, that execute external programs.

2 Examples

In this section, we describe a few example requests and the correct responses for those requests. For each example, assume that your `httpserver` is started in a directory that contains the file, `foo.txt`, with content, "`Hello World, I am foo`", and a file, `bar.txt` containing the content, "`Hello World, I am bar`".

Ex. 1 The client makes a `GET` request by sending this string (remember that `\n` refers to a single newline character and **not** the two separate characters `'\'` and `'n'`.

```
GET /foo.txt HTTP/1.1\r\n\r\n
```

The server does not change any files. It responds to the client by sending the client the following response:

```
HTTP/1.1 200 OK\r\nContent-Length: 21\r\n\r\nHello World, I am foo
```

Ex. 2 The client makes a `PUT` by sending:

```
PUT /foo.txt HTTP/1.1\r\nContent-Length: 21\r\n\r\nHello foo, I am World
```

The server replaces the contents of `foo.txt` with "`Hello foo, I am World`" and responds by sending the client the following content

```
HTTP/1.1 200 OK\r\nContent-Length: 3\r\n\r\nOK\n
```

Ex. 3 The client makes a `PUT` request by sending:

```
PUT /new.txt HTTP/1.1\r\nContent-Length: 14\r\n\r\nHello\nI am new
```

The server creates a new file in its directory, named `new.txt`, with the contents "`Hello\nI am new`". It then responds to the client with the following:

```
HTTP/1.1 201 Created\r\nContent-Length: 8\r\n\r\nCreated\n
```

Ex. 4 The client makes a `GET` request by sending:

```
GET /not.txt HTTP/1.1\r\n\r\n
```

Since `not.txt` does not exist in the server's directory, the server responds with the following:

```
HTTP/1.1 404 Not Found\r\nContent-Length: 10\r\n\r\nNot Found\n
```

Ex. 5 The client makes an invalid request by sending:

```
GET /foo.txt HTTP/1.10\r\nhello*world: value\r\n\r\n
```

This request is invalid because (1) `HTTP/1.10` is an invalid `Version` since `10` is not a single digit number; AND (2) The `header-field` is invalid since the key `hello*world` contains the `*` character. So, the server will respond by sending the client the following content:

```
HTTP/1.1 400 Bad Request\r\nContent-Length: 12\r\n\r\nBad Request\n
```

3 Rubric

We will use the following rubric for this assignment:

Category	Point Value
Makefile	10
Clang-Format	5
Files	5
Functionality	80
Total	100

Makefile Your repository includes a Makefile with the rules `all` and `httpserver`, which produce the `httpserver` binary, and the rule `clean`, which removes all `.o` and binary files. Additionally, your Makefile should use clang (i.e., it should set `CC=clang`), and should use the `-Wall`, `-Wextra`, `-Werror`, and `-pedantic` flags (i.e., it should set `CFLAGS=-Wall -Wextra -Werror -pedantic`).

Clang-Format All `.c` and `.h` files in your repository are formatted in accordance with the `.clang-format` file included in your repository.

Files The following files are included in your repository: `httpserver.c`, `Makefile`, and `README.md`. Your repository should not include binary files nor any object files (i.e., `.o` files), except for the file named `asgn2_helper_funcs.a` (see Resources below). To make it easier for you to maintain tests, you can also include binary files in any directory whose name starts with the phrase `test`.

Functionality Your `httpserver` program performs the functionality described in Assignment Details.

4 Resources

4.1 Testing Tips

As always, we give you a couple of resources for testing:

Autograder An autograder, which you will run locally through the command line. As is typical in this class, you will use the autograder to submit your assignments for grading (see the first page of this assignment). You will have 6 submissions every hour (note, this means that if you start on the assignment early, you will have more submissions!)

Testing Scripts A set of test scripts in the resources repository to check your functionality. You can use the tests to see if your functionality is correct by running them on your Ubuntu 24.04 virtual machine. We provide you with a subset of the tests that we run, but they are useful for you to get started. To run the tests, execute: `./load_repo.sh <path_to_your_asgn2_folder>` on the command line from within `<resources>/asgn2`, where `<path_to_your_asgn2_folder>` is the path to your repository. If you followed our suggested directory structure, this will be `../../<cruzid>/asgn2`. Then, execute `./test_repo.sh` from your assignment 2 directory.

Manual Testing We find that students who succeed on this project create their own tests suites to ensure that they are successful. Below, we articulate a few HTTP clients that you can use for testing:

- You can use a web browser such as Firefox, Google Chrome, or Safari. It is cool to see your code interact with something that you use everyday, so, try it! However, these web browsers are extraordinarily robust: our experience is that such web clients will happily accept responses that are formatted incorrectly.
- RECOMMENDED: To test more rigorously, test with `curl`, a command line browser. For example, to execute a `GET` of the file `foo.txt` on an `httpserver` running on port 1234 and place the output into the file `download.txt`, execute:

  ```
  curl http://localhost:1234/foo.txt -o download.txt
  ```

  Or, execute a `PUT` request that puts the file, `foo.txt`, into the location `new.txt`:

  ```
  curl http://localhost:1234/new.txt -T foo.txt
  ```

  `curl` has a few idiosyncrasies that you might accidentally rely upon in your assignment; we've seen this frequently before. To combat this freuqent issue, we suggest (1) executing `curl` with the flag `a`nd (2) using `cse130_nc`.
- RECOMMENDED: You will find a file, `cse130_nc`, in the resources repository that is our port of a standard Linux utility, `nc` or "netcat". To connect to your server on port 1234, execute `./cse130_nc localhost 1234`. Then, you can type in the text that you wish to send to your server. You can also automate this approach by piping data to `cse130_nc`. For example, to send the `PUT` command listed above to your server, execute:

  ```
  printf "PUT /foo.txt HTTP/1.1\r\nContent-Length: 12\r\n\r\nHello World!"
  | ./cse130_nc localhost 1234
  ```

address already in use If you try to start your server immediately after killing a previous instance of it, you will likely see the following error:

```
httpserver: bind error: Address already in use
```

In this case, restart the server with a different port number. The issue is that the operating system must ensure unique ports are used across the entire system; it often waits to gracefully close ports even after the process that was using them terminates.

4.2 Getting Started and Design Tips

This is a pretty large project and there are multiple ways to get started. We describe an approach below based upon our experience working with students in the past; but you do not *have* to follow this section (that is why it's under Hints!).

Design Process

You should design your server before you start writing any code. Your design should articulate exact function prototypes and struct definitions that you plan to use. You may find it helpful to visualize how your server will function by drawing a picture similar to the "box-and-arrows" pictures that we've been drawing in class. By thoroughly designing your system, you will find that you can avoid many potential bugs, thereby saving you hours!

Keep in mind that your design will probably change as you build your server; sometimes, the act of trying to implement something illuminates how bad of an idea it was. If this happens, we encourage you to "go back to the drawing board" and re-design the server. If you find yourself implementing something that you haven't "designed", then you're doing it wrong.

Step 1: Design your Scaffolding Start thinking about high-level "scaffolding"—essentially, what will your `main` function do? At a high level, `main` will first need to parse command-line arguments and create a listener socket for the port. Then, `main` will need to repeatedly: (1) accept a new connection, (2) perform the logic (i.e., process) that connection, and (3) close that connection ("repeatedly"—sounds like a loop!). You should probably use "top-down" modularization. Make each of these tasks (getting command line arguments, accepting connections, processing connections, and closing connections) a separate function. Don't worry about *how* those functions work, just treat them as a "black box" that performs the task you expect of them!

Step 2: Design the details: Parsing Then, we suggest designing how you will parse a client's request. Here's a high-level thought process of what you might try:

1. **Read enough bytes to ensure that the server will have read the entire `Request-Line` from the request.** Since `httpserver` needs to be efficient, it will need to read multiple bytes at a time and place them into a buffer.
   There is one important difference between a socket and a file descriptor that matters for this step. Recall that `read` returns 0 when a program reaches the end of a file. In contrast, a client will not always tell your server that it is done sending data. So, you might find yourself in a case where there is no new data to `read` from the socket, but the socket nonetheless does not return 0.
2. **Identify each of the fields within the buffer** The server should identify each of the fields that are contained in the buffer that you just read. You need to decide where to store each of them; we suggest making a struct.
   No matter where you put the fields, you should ensure that the lifetime of the data (the time that it is allocated, either through `malloc` or by virtue of being on the stack) covers the entire time that the fields are used. In other words: if you use a field in a function, `foo`, then the field must be allocated either (1) dynamically with `malloc`, `calloc`, etc. or (2) on the stack of a function, `bar` that either called `foo`, or calls a function that calls `foo`, or calls a function that calls a function that calls `foo`...

Step 3: Design the details: Processing Finally, you should think about how to actually perform a get/put request logic. Take a look at the helper library to see if any of the things that we provide will help you.

Implementation Approach

Below, we describe steps that can be used to implement your server.

Step 1: Build an Echo Server We suggest that you start building your server by building the Scaffolding. The scaffolding can work by simply repeating the bytes that the client writes to it. This ensures that your code can listen, accept, read from, and close connections.

Step 2: Sending Responses Next, we suggest that you design code that allows your server to send a response. This is important because it will be impossible to test whether your system can parse requests without being sure that your server can produce responses. Our suggestion is to unit test this: use the `cse130_nc` script to validate that your server produces correct responses before you move on to full-scale testing.

Step 3: Request Logic Then, we suggest that you start building code that performs `GET` and `PUT` requests. You might look back at your `memory` assignment and use our helper functions. At this point, you should be able to use any of the tests in Resources that handle "happy-path" scenarios. In other words: your code should work for tests that use requests that are supposed to return a `Status-Code` of 200 or 201.

Step 4: Parsing Requests Finally, we suggest that you start building code parses requests. This is the most complex task in this assignment and the one that trips up the most students. 'C' provides many ways to parse strings, but we find that students are most successful when using the `regex.h` library. *Hint: You can probably use many of the formats that we provided above directly!* After this step, your server should be able to support all of the tests in Resources.

4.3 The Assignment2 Library

We provide you with a set of libraries to help you with a few basic things. The overall goal is to provide you with features that allow you to focus on the learning objectives of the course without being overly bogged down with details that are not important. We also release this libraries to eliminate double jeopardy between the assignments—in many cases the things that we release here are functions that we expect you would have written in your assignment 1.

We note that the libraries *do not* do all of the basic things that you will want in this assignment and they are *not always* applicable for your task at hand. For example, there is at least one instance where you will want to execute `read` directly rather than using our helper functions.

There are two types of things in the helper library. First, we provide three header files, `iowrapper.h`, which includes code for wrapping `read` and `write` to reduce some of the issues around having to call them in while loops (i.e., these implement the code that we discussed in the Unix I/O practicum); `listener_socket.h`, which will wrap your socket use in a more convenient library; and `protocol.h`, which includes many of the building blocks for regular expressions that are written throughout this document.

Second, we provide a linux archive file, `asgn2_helper_funcs.a`. This archive includes each of the function definitions in a binary format. You can treat `asgn2_helper_funcs.a` as if it were a `.o` file in your makefile. For example, to build an executable, `httpserver`, using the files `httpserver.o` and `asgn2_helper_funcs.a`, you might execute:

```
clang -o httpserver httpserver.c asgn2_helper_funcs.a
```

Tips Here are some tips on using the starter libraries:

1. **Do not edit the header files!** The autograder places the original shared library in place when grading your code. Thus, any edits that you make to them will be replaced by the autograder. Instead, create new headers if/when you want to add new declarations to your code.
2. **Only use pointers to struct Listener_Socket** The starter code *does not* expose a definition of `struct Listener_Socket`, so your program will not be build if you try to declare a variable of that type. You're falling into this trap if you see either of the following compilation errors:

   ```
   error: variable has incomplete type 'struct Listener_Socket'
   error: variable has incomplete type 'Listener_Socket_t' (aka 'struct Listener_Socket')
   ```

   Instead, you should use pointers. Declare `struct Listener_Socket*` variables that you create with `ls_new` and destroy with `ls_delete`.